

Measuring Energy debt

Hugo Simões Marques
University of Minho
Braga, Portugal
pg60266@uminho.pt

Abstract

To investigate whether technical debt reduction improves energy consumption, I systematically searched multiple datasets for projects with substantial debt changes. After evaluating over 450 projects and applying strict filtering criteria, I measured energy consumption in two Apache Foundation projects using RAPL hardware counters across 30 test suite executions each.

Contrary to expectations, reducing technical debt did not decrease energy consumption. Both projects showed increased energy and execution time in their low-debt versions despite 28-50% debt reductions. This suggests technical debt and energy consumption are largely orthogonal. I discuss possible explanations and conclude that energy efficiency and code quality improvements should not be assumed to be aligned.

Keywords: energy debt, technical debt, energy measurement, RAPL, software refactoring, code quality

ACM Reference Format:

Hugo Simões Marques. 2026. Measuring Energy debt. In *Proceedings of Project in Software Development, Validation and Evolution (PSDVM '26)*. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

1 Introduction

Energy consumption has become an increasingly important concern in software engineering, driven by the proliferation of mobile devices, data centers, and environmentally conscious development practices. While software performance has traditionally been measured in terms of execution speed and memory usage, energy efficiency represents a different but equally important dimension.

Couto et al. introduced the concept of energy debt[2], defining it as the implied cost in terms of energy consumption over time of choosing a flawed implementation of a software system. The concept parallels technical debt: just as

technical debt represents the accumulated cost of poor code quality decisions, energy debt represents the long-term energy penalty incurred by suboptimal implementations. This framing suggests that reducing technical debt might simultaneously reduce energy consumption, creating a potential co-benefit of refactoring efforts.

However, despite the intuitive appeal of this relationship, there is limited empirical evidence demonstrating whether technical debt actually correlates with increased energy consumption. Most research on technical debt focuses on maintainability, code quality, and development velocity. Similarly, energy efficiency studies typically examine algorithm optimization and hardware utilization, rather than code quality metrics. The intersection of these two domains remains largely unexplored.

This gap motivated the present study: to investigate whether technical debt reduction leads to measurable improvements in energy consumption. I systematically searched multiple datasets for projects with measurable technical debt changes, ultimately measuring energy consumption across several Apache Foundation projects using hardware-level monitoring. My findings challenge the assumed relationship between technical debt and energy, revealing that the two metrics may be largely orthogonal.

2 Methodology

To study energy debt, we need similar versions of the same project with considerable differences in technical debt. Finding such projects is complicated, most projects don't undergo one clean refactoring that leaves everything exactly the same and even if they did, finding them is complicated. I went through several methods of procurement:

2.1 Research Papers

There is a plethora of published papers on the subject of Technical Debt, going over topics such as SATD (Self Admitted Technical Debt), the use of LLM's in the removal of Technical Debt and empirical studies on Refactorings.

Due to that, I first started looking through ACM DL[1] for papers that included eligible pairs to conduct my research on. Either due to my lack of familiarity with the website or its overall lack of filters on the free version, I was unable to find anything useful.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

PSDVM '26, 2025-2026, University of Minho, Portugal

© 2026 Copyright held by the owner/author(s).

ACM ISBN 978-x-xxxx-xxxx-x/YY/MM

<https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

2.2 Pull Requests

GitHub is where the majority of code ends up at, and the existence of open-source programs means there are several commits and pull requests that we can examine to find major refactorings.

While I was able to find several pull requests with potential to be useful, the lack of confidence in them and sheer amount of ones to examine led me to move on to other methods.

2.3 The Technical Debt Dataset

After failing to find any projects I could work with, I stumbled upon The Technical Debt Dataset, Version 2.0[3] which consists of the analysis of 30 Apache Java projects analyzed with SonarQube across their complete git histories, containing over 900,000 commits and comprehensive metrics on code smells, bugs, vulnerabilities, and, more importantly, technical debt evolution.

To analyze a dataset of this size, I first mapped the tech debt to the date, creating a timeline of each project's debt.

As we can see in Figure 1, several projects have massive dips in total hours, with some losing thousands of hours in a short amount of time. I looked into each of the good drops of debt, but most of them were either massive codebase removals or full rewrites.

So instead, I mapped debt lost between commit pairs and the percentage of lines of code changed, as can be seen in Figure 2.

After filtering out projects that were either too old or too big to be reasonably executed, I ended up with the following:

Project	Debt Reduction (%)	Debt Removed (h)	LOC Changed (%)
Thrift	37.0	119.6	0.04
Configuration	15.3	76.4	0.26
Httpcore	13.1	93.6	0.03
Digester	8.4	37.7	0.01
Collections	2.9	22.7	0.06

Table 1. Selected projects from the Technical Debt Dataset.

However, after compiling the code I ran into several issues that prevented the projects from being examined, such as:

- Ancient Java versions: projects targeted Java 5, which is incompatible with modern systems and impossible to run reliably;
- Obsolete libraries: required dependencies no longer exist or are unmaintained;
- Inaccessible dependencies: required packages are no longer publicly available.

Only Httpcore and Digester successfully compiled, but after using SonarQube to test the tech debt, I found:

Project	Listed Debt Change	Found Debt Change
Httpcore	-93.6h	+44h
Digester	-37.7h	0h

Table 2. Listed and found debt changes after recompilation and SonarQube analysis.

So overall, an increase in debt in one case and the same amount in another. Because of this and the difficulty in compiling the projects to begin with, I decided to try other methods.

2.4 SQuaD

SQuaD: The Software Quality Dataset[4] includes software quality metrics and technical debt data from 450 open-source projects, covering both recent and established projects across their complete development histories.

To identify suitable commit pairs for energy analysis, I filtered the SQuaD dataset using the following criteria:

First, I removed all releases with technical debt below 1,000 minutes, as these likely represented empty or stub releases. For each project with remaining data, I identified the release with the highest and lowest technical debt, creating a high-debt/low-debt pair.

To ensure both snapshots represented substantial codebases, I applied a ratio filter: the low-debt release had to contain at least 20% of the high-debt release's technical debt. I also applied a threshold to eliminate pairs with extreme debt reductions (greater than 80%), which often indicated large-scale code removal or complete rewrites rather than genuine refactoring.

From the resulting pairs, I calculated the technical debt reduction (delta in minutes) and the ratio of low to high debt. Pairs were then sorted by the magnitude of debt reduction to prioritize those with the most substantial improvements.

As shown in Figure 3, four projects had sufficient debt variation to be selected for energy measurement: maven-enforcer, commons-csv, commons-email, and commons-logging. These projects showed significant debt reductions (ranging from 1,700 to 2,000 minutes) while maintaining substantial codebases in both high and low debt states.

Just like before, maven-enforcer and commons-logging failed to compile for similar reasons and were ignored, the remaining two had a real debt change, as seen in Table 3.

Project	Listed Debt Change (%)	Found Debt Change (%)
commons-csv	-52.9	-83.6
commons-email	-61.3	-83.1

Table 3. Listed and found debt changes for selected SQuaD projects.

Now that we have some projects with real debt drop, let's measure the energy cost of before and after.

2.5 Energy Measurement

To measure energy consumption, I used RAPL (Running Average Power Limit), which is built into Intel processors and provides direct access to energy counters via CPU registers. This approach offers sub-joule precision without requiring external hardware.

For each execution, I recorded multiple energy metrics including total package energy (entire CPU socket), core energy (CPU cores and caches), and execution time in milliseconds. I also monitored CPU temperature to identify any thermal throttling that could affect results.

The measurement procedure for each project version was as follows: I began with a 30-second idle period to allow the CPU to reach a stable baseline temperature. Between each of the 30 executions, I waited for the CPU to cool back to within 3 degrees of the baseline temperature before starting the next run, preventing thermal throttling from contaminating later measurements. Energy was recorded as the difference in RAPL counter values before and after each execution.

To ensure consistency across all measurements, I applied strict controls: all runs used the same hardware, operating system, and Java Virtual Machine (OpenJDK 8). I fixed the MAVEN_OPTS parameter and closed unnecessary background processes during measurements.

2.6 Round 1: Manual Selection

With the energy measurement set up, all that remained was to measure it and examine the results.

As we can see in Figure 4, the results are very different between project, showing a massive decrease in energy in commons-csv and a significant increase in commons-email. The same can be observed in the time it took to run the test suite.

After some examining, it appears that the Before and After are that are quite a bit apart, leading to drastically different tests and, therefore, results. To fix this, I matched the tests, by name, and redid the measurements.

As shown in Figure 5, these results are significantly better, but show an increase in both time and energy between versions in both cases, which is not the expected outcome.

Due to the versions being so far apart and the results inconclusive, I moved on to a new round of SQuaD projects.

2.7 Round 2: Automated Filtering

To address the issues in Round 1, I developed a more rigorous approach to select project pairs. I implemented automated scanning of the SQuaD dataset to identify consecutive releases meeting strict technical criteria: maximum 25% NCLOC change, releases within one year of each other, and at least 30% technical debt reduction.

However, automated filtering alone was insufficient. The candidates included data artifacts and unreliable comparisons. I manually reviewed the results and applied additional domain-specific filters: removing major version jumps, prioritizing patch releases and release-candidate-to-final transitions, and discarding suspicious entries with implausible version sequences.

As shown in Figure 6, the refined selection yielded four strong candidates in the preferred zone. To ensure both the debt and the LOC change were accurate, I manually verified both:

Project	Listed LOC Change (%)	Found LOC Change (%)	Listed Debt Change (%)	Found Debt Change (%)
spark_1.2	0.0	+0.75	-46.8	+0.23
spark_3.3	0.0	+0.11	-43.0	+0.19
arrow-rs_4	-4.32	+0.32	-89.0	+0.48
ant_ivy	-0.06	-0.31	-42.0	+0.77

Table 4. Listed and found LOC and debt changes for Round 2 candidates.

While the LOC change stayed roughly the same, the found debt is nothing like the listed one, with the worst case being arrow-rs_4, with a listed -89.0% debt reduction while the found one was +0.48%.

Since all four of these projects had little to no real debt drop, I decided to move on to another round.

2.8 Round 3: Direct Measurement

The consistent inconsistency of listed SonarQube measurements and the found measurements can only be explained by a difference in versions and rules of SonarQube. Due to this, in this round I opted to measure the debt myself on every possible project.

First I filtered the projects on looser criteria, maximum 30% NCLOC change and releases within 18 months of each other.

With a large amount of pairs to work with, I built an automated pipeline to measure technical debt directly for each candidate. For each project pair, the process consisted of four steps:

1. Clone the repository;
2. Build both versions;
3. Scan with SonarQube;
4. Record the results.

As we can see in Figure 7, only 16 projects were properly measured, a lot of pairs had at least one of the versions fail to compile.

From the 16, 10 presented 0 real debt change, 2 had the debt actually increase and only 4 had a real drop in tech debt. Of these four, the pair of commons-imaging contained an alpha version and directory-kerby had too small of a real debt drop to be considered.

Project	Listed LOC Change (%)	Found LOC Change (%)	Listed Debt Change (%)	Found Debt Change (%)
shardingsphere	+7.2	+1.8	-42.8	-50.0
maven-enforcer	+9.2	+21.4	-31.2	-28.6

Table 5. Listed and found LOC and debt changes for selected Round 3 projects.

Maven-enforcer had a significant increase in lines of code but it is still reasonable for a full refactoring.

With two good projects ready and compiled, we move on to the measurements, although the tests were already similar, I still matched the test names to ensure fair results.

The measurements can be seen in Figure 8 and much like with the results in Figure 5, there is an increase in both energy and time between high and low debt versions, with the J/ms being roughly the same.

3 Results and Discussion

The measurements revealed that reducing technical debt did not lead to decreased energy consumption. In both projects, the low-debt versions consumed more energy and time than their high-debt counterparts. This suggests a weak or non-existent correlation between technical debt and energy efficiency. Even in the less certain Figure 5, this correlation, or lack thereof, is present.

This could be due to many reasons, such as improper refactoring, where code to do the same thing ended up more complex, the versions still including other changes that aren't purely debt removals or even the type of technical debt removed may be more important than the quantity, as different debt categories may have different energy implications.

4 Conclusions

While a direct correlation wasn't found, it doesn't mean one might not exist, just probably depends on the type of debt that is being evaluated. The best way to measure energy debt would be to start with a project full of debt, categorize it, remove it and analyze each kind individually as well as trying to stay as faithful to the original code and it's logic as possible.

A critical finding from this research is that SonarQube's metrics are time-dependent. The gap between listed and

found debt changes demonstrates that datasets become unreliable when the analysis tools' rules have evolved significantly.

5 Attachments

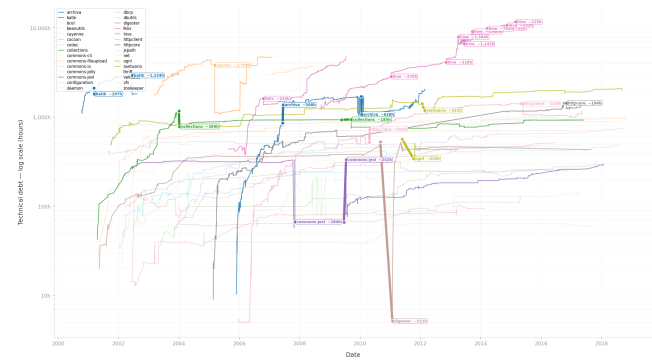


Figure 1. Technical debt timeline mapped from hours of debt to date.



Figure 2. Technical debt removed mapped from lines of code changed. Pairs selected with a star.

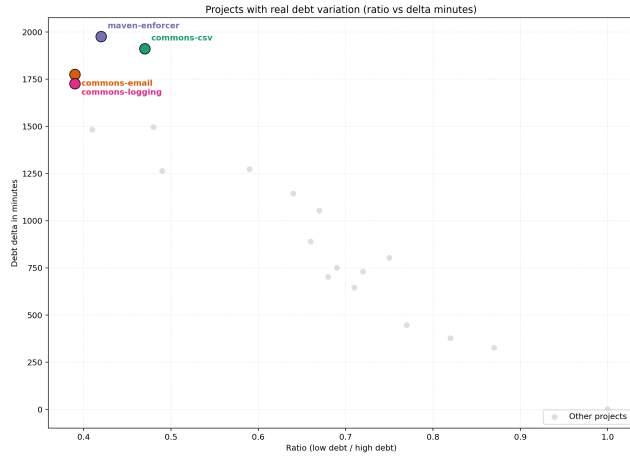


Figure 3. Technical debt delta to ratio between high and low debt commitment pairs.

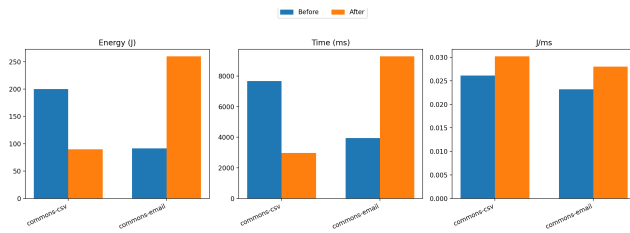


Figure 4. Energy, time and energy per time of commons-csv and commons-email before and after.

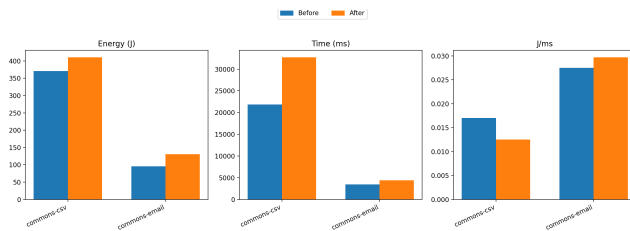


Figure 5. Energy, time and energy per time of commons-csv and commons-email with test matching.

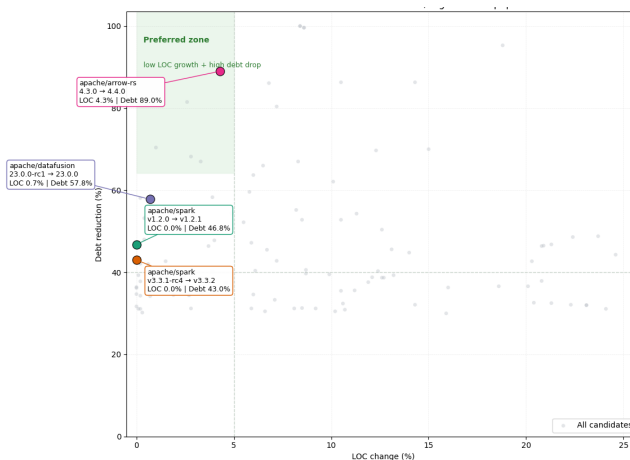


Figure 6. Candidate pairs from Round 2 filtering. The preferred zone represents low LOC change with high debt reduction.

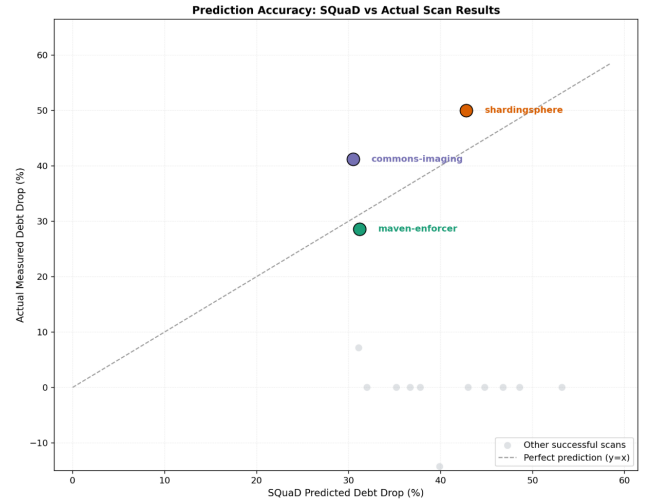


Figure 7. Prediction accuracy of SQuAD debt metrics vs. actual measured debt.

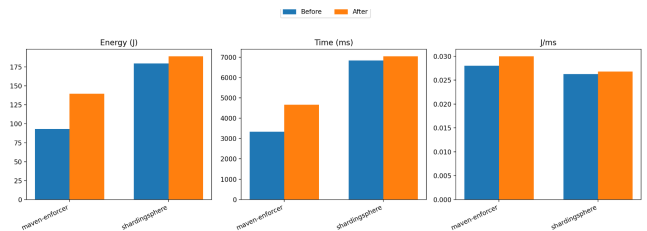


Figure 8. Energy, time and energy per time of maven-enforcer and shardingsphere before and after.

References

- [1] ACM. 2024. *ACM Digital Library*. <https://dl.acm.org/>
- [2] Marco Couto, Daniel Maia, João Saraiva, and Rui Pereira. 2020. Energy Debt. In *Proceedings of the 3rd International Conference on Technical Debt*. 62–66. <https://doi.org/10.1145/3387906.3388628>
- [3] Valentina Lenarduzzi, Nytyi Saarimäki, and Davide Taibi. 2019. The Technical Debt Dataset. In *15th Conference on Predictive Models and Data Analytics in Software Engineering*.
- [4] Mikel Robredo, Matteo Esposito, Davide Taibi, Rafael Peñaloza, and Valentina Lenarduzzi. 2025. SQuAD: The Software Quality Dataset. *arXiv preprint arXiv:2511.11265* (2025).